

DEPTH FIRST SEARCH

AIM:

To explore all vertices of a graph or tree by traversing as far as possible along each branch before backtracking.

```
1  def dfs(graph, start, visited=None):
2      if visited is None:
3          visited = set()
4          visited.add(start)
5          print(start, end=' ')
6          for neighbor in graph.get(start, []):
7              if neighbor not in visited:
8                  dfs(graph, neighbor, visited)
9
10     # User input
11     graph = {}
12     nodes = int(input("Enter number of nodes: "))
13     for _ in range(nodes):
14         node = input("Enter node name: ")
15         neighbors = input(f"Enter neighbors of {node} (space-separated): ").split()
16         graph[node] = neighbors
17
18     start_node = input("Enter start node for DFS: ")
19     print("DFS Traversal:")
20     dfs(graph, start_node)
```

RESULT:

A traversal order of nodes (or discovery of a path, component, or solution) depending on the problem, such as a list of visited nodes or detection of cycles, connected components, or paths.